

Содержание

План эксперимента: дополнительная метрика для последних итераций	1
1. Цель	1
2. Базовый key recovery	1
3. Prefix-максимумы до 7 знака	2
4. Хвостовая схема	2
5. True-метрика	2
6. False-метрики	2
7. Сравнение с существующими метриками	3
8. Экономия времени	3
9. Что доработать в коде	4
10. Проверки перед большим запуском	4
11. Оценка времени	4
12. Параллельный запуск	4

План эксперимента: дополнительная метрика для последних итераций

1. Цель

Нужно запустить прежние серии key recovery и добавить к ним новую метрику, которая объясняет поведение ложных и истинных дельт через произведение двух дельт на хвостовой части шифра.

Основные серии остаются прежними:

```
adaptive_m1    = 1 / delta^2
adaptive_m5    = 5 / delta^2
adaptive_m10   = 10 / delta^2
full_material  = полный материал
```

При этом серии должны идти на разных наборах ключей. Для базового SEED используем разные seed:

```
adaptive_m1     seed = SEED + 4000037
adaptive_m5     seed = SEED + 5000011
adaptive_m10    seed = SEED + 6000011
full_material   seed = SEED + 7000003
```

Таким образом сравнение между сериями будет статистическим, а не построчным по одному и тому же ключу.

2. Базовый key recovery

Для каждого 32-битного истинного ключа K1K2K3K4 обычный эксперимент делает следующее:

1. Строит преобразование первых 10 итераций.
2. Ищет максимальную линейную корреляцию:
 $\alpha \rightarrow \beta$
3. По найденному $\delta_{\text{prefix}} = |\delta(\alpha, \beta)|$ выбирает объем материала:
 $N = \lceil m / \delta_{\text{prefix}}^2 \rceil$
где m равно 1, 5 или 10 для адаптивных серий. Для `full_material` используется:
 $N = 2^{16}$
4. Перебирает кандидатов последних ключевых байтов.
5. Считает старые метрики:
 $|\delta_{\text{true}}|$
 $\text{mean } |\delta_{\text{false}}|$
 $\text{max } |\delta_{\text{false}}|$
true rank
top1 / top3 / top32

3. Prefix-максимумы до 7 знака

Для новой хвостовой метрики нужно использовать не только один канонический максимум первых 10 итераций.

После поиска по первым 10 итерациям берем все пары (α , β), которые достигают максимума $|\delta_{\text{prefix}}|$ с точностью до 7 знака:

```
M = {
  ( $\alpha_i$ ,  $\beta_i$ ,  $\delta_i$ )
  :
  round( $|\delta_i|$ , 7) = round(max  $|\delta_{\text{prefix}}|$ , 7)
}
```

Для каждого такого β_i отдельно считаем хвостовую метрику. Затем итоговые значения по ключу выбираются как максимум по всем β_i из множества M.

4. Хвостовая схема

Для каждого β_i рассматриваем хвост:

```
 $\beta_i$ 
-> 11-я и 12-я итерации по истинному ключу
-> T
-> кандидатная часть по ложным ключам
-> output mask
```

Где T - перестановка, которая стоит после первой хвостовой таблицы. Ее нужно обязательно учитывать между первой и второй частью.

Условно:

```
A -- true key --> B -- T --> B_T -- candidate key --> C
```

Первая часть считается только по истинному ключу текущего большого ключа. Здесь ключи не перебираются.

Вторая часть считается по кандидатным ключам. Для false-метрик перебираются все кандидатные пары, кроме истинной.

5. True-метрика

Для истинной пары последних ключевых байтов считаем:

```
LAST4_TRUE_PRODUCT( $\beta_i$ )
=
max over middle/output masks (
   $|\delta_{1\text{true}}(\beta_i, \text{middle})|$ 
  *
   $|\delta_{2\text{true}}(T(\text{middle}), \text{output})|$ 
)
```

Для итоговой строки сохраняем:

```
LAST4_TRUE_PRODUCT_VALUE
LAST4_TRUE_DELTA1_VALUE
LAST4_TRUE_DELTA2_VALUE
LAST4_TRUE_MIDDLE_MASK
LAST4_TRUE_MIDDLE_MASK_AFTER_T
LAST4_TRUE_OUTPUT_MASK
```

Если prefix-максимумов несколько, итоговый LAST4_TRUE_PRODUCT_VALUE берется как максимум по всем β_i .

6. False-метрики

Для каждого ложного кандидата последних ключевых байтов:

```
product_false(candidate,  $\beta_i$ )
=
max over middle/output masks (
```

```

|delta1_true(beta_i, middle)|
*
|delta2_false_candidate(T(middle), output)|
)

```

Истинная кандидатная пара из false-перебора исключается.

После этого считаем:

```

LAST4_FALSE_PRODUCT_MAX(beta_i)
=
max over false candidates product_false(candidate, beta_i)

```

```

LAST4_FALSE_PRODUCT_MEAN(beta_i)
=
mean over false candidates product_false(candidate, beta_i)

```

Если prefix-максимумов несколько, итоговый LAST4_FALSE_PRODUCT_MAX_VALUE берется как максимум по всем beta_i. Среднее LAST4_FALSE_PRODUCT_MEAN_VALUE сохраняется для того beta_i, на котором был достигнут выбранный false-максимум. Это соответствует логике поиска максимального произведения.

Для строки сохраняем:

```

LAST4_FALSE_PRODUCT_MAX_VALUE
LAST4_FALSE_PRODUCT_MEAN_VALUE
LAST4_FALSE_MAX_KEY_PAIR
LAST4_FALSE_MAX_DELTA1_VALUE
LAST4_FALSE_MAX_DELTA2_VALUE
LAST4_FALSE_MAX_MIDDLE_MASK
LAST4_FALSE_MAX_MIDDLE_MASK_AFTER_T
LAST4_FALSE_MAX_OUTPUT_MASK

```

7. Сравнение с существующими метриками

Добавляем отношения:

```

LAST4_FALSE_TRUE_PRODUCT_RATIO
= LAST4_FALSE_PRODUCT_MAX / LAST4_TRUE_PRODUCT

```

```

LAST4_FALSE_MEAN_TRUE_PRODUCT_RATIO
= LAST4_FALSE_PRODUCT_MEAN / LAST4_TRUE_PRODUCT

```

И сравниваем их с уже существующими отношениями:

```

FALSE_DELTA_ABS_MAX / TRUE_DELTA_ABS
MEAN_FALSE_DELTA_ABS / TRUE_DELTA_ABS

```

Цель сравнения - понять, насколько структурная оценка хвоста через произведение двух дельт объясняет наблюдаемое поведение ложных и истинных кандидатов в полном key recovery.

8. Экономия времени

Главная экономия:

1. Не считаем хвост для всех beta.
2. Считаем хвост только для тех beta_i, которые уже дали максимум первых 10 итераций до 7 знака.
3. Вторую часть хвоста можно предрасчитать как lookup-таблицу.

Предрасчитываем таблицу:

```

lookup2[candidate_pair][input_mask_after_T] = {
    best_delta2_abs,
    best_delta2_signed,
    best_output_mask
}

```

Эта таблица не зависит от текущего большого ключа и может переиспользоваться во всех сериях.

Первая часть зависит от истинного ключа и текущего β_i , поэтому ее считаем на лету. Но таких β_i должно быть немного, потому что они ограничены максимумами первых 10 итераций до 7 знака.

9. Что доработать в коде

1. В поиске prefix delta сохранить список всех (α, β), достигающих максимума до 7 знака.
2. Реализовать builder lookup-таблицы для второй хвостовой части.
3. Добавить бинарный формат и загрузку lookup-таблицы.
4. Добавить расчет первой true-части хвоста внутри `key_recovery_experiment`.
5. Добавить расчет true/false хвостовых произведений через lookup.
6. Добавить новые колонки в `summary.csv`.
7. Добавить агрегаты в `aggregate_report.csv`.
8. Обновить генератор отчета, чтобы он явно описывал формулы старых и новых параметров.
9. Проверить учет перестановки T.
10. Проверить, что истинная пара исключается из false-метрик.

10. Проверки перед большим запуском

Перед серверным запуском:

1. Smoke-gun на `count=1`.
2. Проверить, что seed между сериями разные.
3. Проверить, что prefix-максимумы до 7 знака сохраняются корректно.
4. Проверить, что хвостовая часть запускается для каждого β_i из множества prefix-максимумов.
5. Проверить учет перестановки T.
6. Проверить, что true-кандидат исключен из false max и false mean.
7. Проверить несколько строк вручную на малом count.
8. Сделать прогон `count=1000` для оценки реального overhead новой метрики.

11. Оценка времени

Оценка для одного RTX 4090 по данным предыдущего большого прогона:

<code>adaptive_m1</code>	~32 часа
<code>adaptive_m5</code>	~33 часа
<code>adaptive_m10</code>	~34 часа
<code>full_material</code>	~45 часов

Старые 4 серии на одном сервере:

~144 часа

Новая lookup-метрика:

построение lookup-таблицы:	~4-8 часов
добавка хвостовой метрики:	~5-15 часов

Итого на одном RTX 4090:

~153-167 часов
~6.4-7 суток

12. Параллельный запуск

Чтобы не ждать неделю, лучше параллелить серии по нескольким RTX 4090.

Вариант на 4 серверах:

<code>server 1:</code>	<code>adaptive_m1</code>
<code>server 2:</code>	<code>adaptive_m5</code>
<code>server 3:</code>	<code>adaptive_m10</code>
<code>server 4:</code>	<code>full_material</code>

Lookup-таблицу лучше построить один раз и разложить на все серверы. Если это неудобно, можно строить ее на каждом сервере отдельно, но это увеличит GPU-часы.

Оценка wall-clock при 4 серверах:

самая длинная серия:	~45 часов
lookup + новая метрика overhead:	~5-15 часов
итого:	~50-60 часов
	~2-2.5 суток

По деньгам GPU-часы будут близки к одному серверу на 6-7 суток, но ждать придется примерно в 3 раза меньше.